

VISION 6 AI

Project Explanation Document

1. Introduction

Vision 6 AI is a full-stack, AI-powered exam preparation web application. It pairs a lightweight, responsive frontend with a Python FastAPI backend that securely integrates Anthropic's Claude language model. The application turns a single topic or block of notes into five different forms of study material — explanations, summaries, MCQs, quizzes, and mock interview questions — all delivered as live, streamed text rather than a static block returned after a delay.

2. Problem & Motivation

Manually preparing revision material — writing practice MCQs, condensing long notes, building quizzes, rehearsing interview answers — is repetitive and time-consuming. Most students either skip this preparation altogether or rely on generic chatbots with no purpose-built workflow for exam study. Vision 6 AI addresses this by wrapping an LLM in a study-specific interface, so the same input (a topic or a page of notes) can be turned into whichever study format the student needs next.

3. System Architecture

The application follows a standard three-tier architecture: a static frontend client, a backend API server, and an external LLM provider. The backend is the only layer that holds the Anthropic API key, and it streams model output back to the browser using Server-Sent Events (SSE) so that responses render progressively.

Request flow: Browser (JS) → FastAPI backend (/api/... route) → Anthropic Claude API (streaming) → FastAPI streams SSE chunks back → Browser renders text live in the chat window.

Layer	Responsibility
Frontend (HTML/CSS/JS)	Renders UI, captures user input, opens a streaming fetch() request, and displays text as it arrives.
Backend (FastAPI)	Validates requests, holds the API key, calls Claude, relays the token stream to the client, and enforces CORS.
Claude API (Anthropic)	Generates the actual explanation, summary, MCQs, quiz, or interview content.

4. Technology Stack

Component	Technology / Library
Frontend structure	HTML5, CSS3 (responsive.css for mobile), Vanilla JavaScript
Backend framework	FastAPI (Python 3.12)
AI integration	anthropic Python SDK, AsyncAnthropic streaming client
Data validation	Pydantic models (request schemas per endpoint)
Real-time transport	Server-Sent Events (text/event-stream) over HTTP
Config / secrets	python-dotenv, OS-level environment variables
Containerization	Docker (single-stage Python slim image)
Target deployment	AWS App Runner (or Elastic Beanstalk), via ECR or GitHub-connected build

5. Core Modules / Features

5.1 AI Chat

The main conversational interface. User messages and prior turns are sent as a message history to `POST /api/chat/stream`, and Claude's reply streams back token-by-token into the chat bubble in real time.

5.2 Notes Summarizer

Takes raw pasted notes and condenses them into a concise, structured summary (bullet-point, concise, or detailed style) via `POST /api/study/summarize`.

5.3 MCQ Generator

Generates a set of topic-specific multiple-choice questions, each with four options, a correct answer, and a short explanation, via `POST /api/study/mcq`. Difficulty is configurable.

5.4 Quiz Generator

Builds a longer, mixed-format quiz (MCQ, short-answer, true/false) on a given topic via `POST /api/study/quiz`.

5.5 Interview Preparation

Runs an interactive mock interview: the backend asks one question at a time, gives brief feedback on the student's previous answer, and asks the next question, via `POST /api/study/interview`.

5.6 Supporting UI Features

- Word/character counter, quick-prompt buttons, and subject/difficulty selectors on the input form.
- Copy chat, clear chat, and download conversation controls.
- Light/dark theme toggle and a responsive layout for mobile and desktop.
- Toast notifications and a progress bar for feedback during generation.

6. Backend API Reference

Method	Endpoint	Purpose
POST	<code>/api/chat/stream</code>	Streamed conversational AI response
POST	<code>/api/study/explain</code>	Streamed topic explanation
POST	<code>/api/study/summarize</code>	Streamed notes summary
POST	<code>/api/study/mcq</code>	Streamed MCQ set (JSON)
POST	<code>/api/study/quiz</code>	Streamed quiz (JSON)
POST	<code>/api/study/interview</code>	Streamed mock-interview turn
GET	<code>/health</code>	Health check for load balancer / App Runner

All AI endpoints return text/event-stream responses using the format data: {"type": "delta", "text": "..."}, ending with {"type": "done"}.

7. Working / Data Flow

- Student enters a topic, notes, or question into the input field (with optional subject/difficulty).
- Clicking a tool button (Ask AI, Summarize, Generate MCQ, Generate Quiz) triggers a fetch() call from js/api.js to the matching backend endpoint.
- FastAPI validates the request body against a Pydantic model, then opens a streaming call to the Claude API using the server-side ANTHROPIC_API_KEY.
- As Claude generates tokens, FastAPI immediately forwards each chunk to the browser as an SSE event.
- The frontend appends each chunk to the chat bubble live, so the response appears to type itself in real time.
- Once the model finishes, a final 'done' event closes the stream and removes the streaming cursor.

8. Security Implementation

- The Anthropic API key is read only from a server-side environment variable and is never included in any frontend file or response.
- CORS is restricted to an explicit ALLOWED_ORIGINS list rather than left open to all origins.
- Secrets are excluded from version control via .gitignore and from the Docker image via .dockerignore.
- Environment variables are injected at container runtime (Docker/App Runner), never baked into the image.
- All input to the AI endpoints is validated through typed Pydantic models before use.

9. Containerization & Deployment

The backend ships with a Dockerfile that installs dependencies, copies the application code, and runs it with Uvicorn on the port provided by the PORT environment variable — matching how AWS App Runner assigns ports.

```
docker build -t vision6-backend .  
docker run -p 8000:8000 --env-file .env vision6-backend
```

For production, the image (or connected GitHub repo) is deployed to AWS App Runner, which builds/runs the container and automatically provisions a public HTTPS URL. Environment variables (API key, allowed origins, model name) are configured directly in the App Runner service settings rather than the source code.

10. Project Structure

```
VISION_6_IBM_JULY_26/  
  index.html, css/, assets/ – frontend UI  
  js/config.js, api.js – backend URL + SSE streaming helper  
  js/chat.js, app.js – UI logic wired to the backend  
  backend/app/main.py – FastAPI app, CORS, routing  
  backend/app/claude_client.py – Anthropic client setup  
  backend/app/routes/chat.py, study_tools.py – API endpoints  
  backend/Dockerfile, requirements.txt, .env.example – build & config
```

11. Future Enhancements

- User accounts and persistent history stored in a database instead of local state.
- File upload support so students can summarize PDFs or slide decks directly.
- Exportable PDF/Word study packs generated from a session's Q&A.
- Usage analytics dashboard (questions asked, topics covered, weak areas).
- Rate limiting and per-user API quotas for production-scale deployment.

12. Conclusion

Vision 6 AI demonstrates a complete, secure, production-style integration between a web frontend and a large language model: a responsive UI, a FastAPI backend that keeps credentials server-side, real-time streamed responses, and a containerized, cloud-deployable architecture — directly fulfilling the assignment's frontend, backend, streaming, security, and deployment requirements.